빌드 시스템 및 배포 과정 분석

개요

Oasis CLI는 Go 언어로 작성된 명령줄 도구로, 현대적인 빌드 시스템과 자동화된 배포 파이프라인을 갖추고 있습니다. 이 문서는 Makefile 기반의 빌드 시스템, Go 모듈 의존성 관리, 그리고 GoReleaser를 활용한 배포 과정을 상세히 분석합니다.

빌드 시스템 구조

1. Makefile 기반 빌드 시스템

주요 Makefile 구성

메인 Makefile

- `common.mk` 파일을 포함하여 공통 설정과 유틸리티 함수 활용
- 타겟별로 명확히 구분된 빌드, 테스트, 린트, 정리 작업
- 플랫폼별 빌드 지원 (Linux, macOS, Windows)

주요 빌드 타겟들:

```
# 기본 빌드 (현재 플랫폼)
build: oasis

# Windows용 빌드
build-windows: oasis.exe
```

```
# 메인 바이너리 빌드
oasis: $(shell find . -name "*.go" -type f)  $(shell find cmd/rofl/init_artifact
@$(GO) build -v -o oasis $(GOFLAGS)  $(GO_EXTRA_FLAGS)
```

빌드 플래그 및 최적화

링커 플래그 설정:

- `GOLDFLAGS_VERSION` : 버전 정보를 바이너리에 임베드
- `GO_EXTRA_FLAGS` : 추가 빌드 플래그 지원
- `-trimpath` : 빌드 경로 정보 제거로 재현 가능한 빌드 보장

버전 관리:

```
# Git 태그 기반 버전 결정
GIT_VERSION := $(subst v,,$(shell \
git describe --tags --match 'v*' --abbrev=0 2>/dev/null HEAD || \
echo undefined \
))

# 링커 플래그로 버전 정보 주입
export  GOLDFLAGS_VERSION := -X github.com/oasisprotocol/cli/version.Software=$(
```

2. Go 모듈 시스템

의존성 관리 구조

go.mod 파일 분석:

- Go 1.24.0 이상 요구
- 주요 의존성:
- `github.com/spf13/cobra` : CLI 프레임워크
- `github.com/spf13/viper` : 설정 관리
- `github.com/oasisprotocol/oasis-core/go` : Oasis Core 라이브러리
- `github.com/oasisprotocol/oasis-sdk/client-sdk/go` : Oasis SDK

의존성 교체 (replace 지시문):

```
// 버그 수정이 포함된 포크 사용
replace  github.com/spf13/cast => github.com/oasisprotocol/cast  v0.0.0-20220606


// Oasis 특화 CometBFT 버전 사용
replace  github.com/cometbft/cometbft => github.com/oasisprotocol/cometbft  v0.3
```

의존성 검증 및 관리

자동화된 의존성 검사:

```
# go mod tidy 검증
define  CHECK_GO_MOD_TIDY =
$(GO) mod tidy; \
if [[ ! -z `git status --porcelain go.mod go.sum` ]]; then \
$(PRINT) "$(RED)Error: The following changes detected after running 'go mod tidy
```

```
git diff go.mod go.sum; \

exit 1; \

fi

endef
```

3. 크로스 플랫폼 빌드 지원

지원 플랫폼

- **Linux**: AMD64, ARM64
- **macOS**: AMD64, ARM64 (Universal Binary 지원)
- **Windows**: AMD64

플랫폼별 컴파일러 설정

```
# .goreleaser.yaml에서 플랫폼별 컴파일러 지정

builds:

- id: oasis-linux-amd64

env:

- CC=x86_64-linux-gnu-gcc

- CXX=x86_64-linux-gnu-g++

- id: oasis-darwin-arm64

env:

- CC=oa64-clang

- CXX=oa64-clang++
```

배포 과정 분석

1. GoReleaser 기반 배포 시스템

배포 설정 구조

GoReleaser 설정 (.goreleaser.yaml):

- 다중 플랫폼 바이너리 빌드
- 아카이브 생성 및 체크섬 계산
- GitHub 릴리스 자동 생성
- 변경 로그 자동 생성

빌드 매트릭스:

```yaml
builds:
- id: oasis-linux-amd64
- id: oasis-linux-arm64
- id: oasis-darwin-amd64
- id: oasis-darwin-arm64
- id: oasis-windows-amd64
```

Universal Binary 생성

```yaml
universal_binaries:
- id: oasis-darwin-universal
ids:
- oasis-darwin-amd64
```

```
  - oasis-darwin-arm64

  replace: true
```

2. CI/CD 파이프라인

GitHub Actions 워크플로우

린트 워크플로우 (ci-lint.yml):

- 코드 품질 검사 (golangci-lint)
- 문서 린트 (markdownlint)
- Git 커밋 메시지 검사 (gitlint)
- Go 모듈 정리 상태 검증

테스트 워크플로우 (ci-tests.yml):

- 다중 OS 테스트 (Ubuntu, macOS, Windows)
- 빌드 검증
- 단위 테스트 실행
- 예제 생성 및 검증

릴리스 워크플로우 (release.yml):

- 태그 푸시 시 자동 트리거
- GoReleaser를 통한 자동 빌드 및 배포
- GitHub 릴리스 생성

워크플로우 트리거 조건

```
# 브랜치 푸시 및 PR에서 트리거
```

```
on:
push:
branches: [master, stable/*]
pull_request:
branches: [master, stable/*]


# 릴리스는 버전 태그에서만 트리거
on:
push:
tags: ['v[0-9]+.[0-9]+*']
```

3. 버전 관리 시스템

버전 결정 로직

Git 기반 버전 관리:

```
# 정확한 태그에 있는 경우 태그 버전 사용
# 그렇지 않으면 태그-git커밋+dirty 형태로 버전 생성
VERSION := $(or \
$(and $(call eq,$(GIT_COMMIT_EXACT_TAG),YES), $(GIT_VERSION)), \
$(shell git describe --tags --abbrev=0)-git$(shell git describe --always --match
)
```

런타임 버전 정보

version 패키지 구조:

```
var (
// 링커에 의해 설정되는 소프트웨어 버전
Software = "0.0.0-unset"
// Go 컴파일러 버전
Toolchain = runtime.Version()
)

// 의존성 버전 동적 조회
func GetOasisSDKVersion() string {
bi, ok := debug.ReadBuildInfo()
// ... 빌드 정보에서 의존성 버전 추출
}
```

품질 보증 시스템

1. 코드 품질 검사

린트 도구 통합

- **golangci-lint**: Go 코드 정적 분석
- **markdownlint**: 문서 형식 검사
- **gitlint**: 커밋 메시지 규칙 검증

린트 설정 파일들

- `.golangci.yml` : Go 린터 설정
- `.markdownlint.yml` : Markdown 린터 설정
- `.gitlint` : Git 커밋 메시지 규칙

2. 테스트 시스템

테스트 타겟 구조

```
test-targets := test-unit
```

```
test-unit:
@$(GO) test -v -race ./...
```

```
test: $(test-targets)
```

다중 플랫폼 테스트

- Ubuntu, macOS, Windows에서 동시 테스트
- 레이스 컨디션 검사 (`-race` 플래그)
- 캐시 최적화를 통한 빌드 시간 단축

3. 예제 검증 시스템

자동 예제 생성

```
# scripts/gen_example.sh를 통한 예제 자동 생성
examples: $(EXAMPLES)
```

```
examples/%.out:  examples/%.in  oasis  scripts/gen_example.sh
@scripts/gen_example.sh $< $@
```

예제 검증 프로세스

1. 입력 파일 (`.in`)에서 명령어 추출
2. 실제 CLI 실행하여 출력 생성
3. 예상 출력 (`.out`)과 비교
4. Git diff를 통한 변경사항 검증

배포 아티팩트

1. 바이너리 아카이브

아카이브 형식

```
archives:
- name_template: "{{replace .ProjectName \" \" \"_\" | tolower}}_{{.Version}}_{{
wrap_in_directory: true
format_overrides:
- goos: windows
format: zip  # Windows는 ZIP, 나머지는 tar.gz
```

체크섬 생성

```
checksum:
name_template: SHA256SUMS-{{.Version}}.txt
algorithm: sha256
```

2. 릴리스 노트 자동 생성

변경 로그 분류

```
changelog:
groups:
- title: Breaking changes
regexp: "^.*breaks[(\\w)]*:+.*$"
- title: New Features
regexp: "^.*feat[(\\w)]*:+.*$"
- title: Bug fixes
regexp: "^.*fix[(\\w)]*:+.*$"
```

개발 환경 설정

1. 필수 도구

개발 도구 요구사항

- **Go 1.24.x**: 메인 개발 언어
- **Make**: 빌드 자동화

- **Git**: 버전 관리
- **golangci-lint**: 코드 품질 검사
- **Node.js**: 문서 린트 (markdownlint)
- **Python**: Git 커밋 검사 (gitlint)

선택적 도구

- **GoReleaser**: 릴리스 빌드 (로컬 테스트용)
- **Docker**: 컨테이너 기반 빌드 환경

2. 로컬 개발 워크플로우

기본 개발 사이클

```
# 1. 의존성 설치 및 정리
go mod tidy

# 2. 코드 포맷팅
make fmt

# 3. 린트 검사
make lint

# 4. 빌드
make build
```

```
# 5. 테스트
make test

# 6. 예제 검증
make examples
```

크로스 플랫폼 빌드 테스트

```
# Windows용 빌드 테스트
make build-windows

# 릴리스 빌드 테스트 (GoReleaser 필요)
make release-build
```

보안 및 재현성

1. 빌드 보안

보안 조치

- **trimpath**: 빌드 경로 정보 제거
- **buildid 제거**: 재현 가능한 빌드 보장
- **체크섬 검증**: 배포 아티팩트 무결성 보장

의존성 보안

- `go.sum` 파일을 통한 의존성 체크섬 검증

- 정기적인 의존성 업데이트 및 보안 패치

2. 재현 가능한 빌드

빌드 환경 표준화

- Docker 컨테이너 기반 릴리스 빌드
- 고정된 Go 버전 및 도구 체인
- 환경 변수 표준화

버전 추적

- Git 커밋 해시 기반 버전 생성
- 빌드 시점의 의존성 버전 기록
- 빌드 메타데이터 임베딩

성능 최적화

1. 빌드 성능

캐시 활용

- Go 모듈 캐시 활용
- GitHub Actions 캐시 최적화
- 증분 빌드 지원

병렬 처리

- 다중 플랫폼 빌드 병렬 실행
- 테스트 병렬 실행 (`-race` 플래그와 함께)

2. 배포 최적화

바이너리 최적화

- 링커 플래그를 통한 크기 최적화
- 불필요한 디버그 정보 제거
- 압축 최적화

확장성 및 유지보수성

1. 빌드 시스템 확장

새로운 플랫폼 추가

1. `.goreleaser.yaml` 에 새 빌드 타겟 추가
2. 필요시 크로스 컴파일러 설정
3. CI/CD 파이프라인에 테스트 추가

새로운 빌드 타겟 추가

1. `Makefile` 에 새 타겟 정의
2. 의존성 및 빌드 규칙 설정
3. `.PHONY` 타겟에 추가

2. 배포 프로세스 개선

배포 채널 확장

- 패키지 매니저 지원 (Homebrew, Chocolatey 등)
- 컨테이너 이미지 배포
- 자동 업데이트 메커니즘

모니터링 및 분석

- 빌드 시간 추적
- 배포 성공률 모니터링
- 사용자 피드백 수집

이 빌드 시스템과 배포 과정은 현대적인 Go 프로젝트의 모범 사례를 따르며, 자동화, 보안, 재현성을 중시하는 설계로 구성되어 있습니다. 지속적인 통합과 배포를 통해 안정적이고 신뢰할 수 있는 소프트웨어 배포를 보장합니다.